

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Repository scope

This directory (`client/`) is the React SPA portion of the Pollux/YPM fork of [project-lux/lux-frontend](#) — customizations of the Yale LUX frontend for the Yale Peabody Museum. The sibling `../server/` directory is the Node server that hosts the built bundle in prod; `../docker/` builds the deployed image. All day-to-day frontend work happens in `client/`.

Commands

Package manager is **yarn** (classic) — `yarn.lock` is authoritative; do not introduce `package-lock.json`.

- `yarn start` — Vite dev server on <http://localhost:3000> (opens a browser). Uses `client/.env` directly.
- `yarn build` — TypeScript check + production build into `dist/`. **Wraps the build in** `mv .env _env / mv _env .env` so the local `.env` does not get compiled in; the server-hosted build receives its config at runtime from the Node server's `/env` endpoint. Don't work around this by removing the rename.
- `yarn preview` — serves the built `dist/` locally.
- `yarn lint` — ESLint 9 flat config (`eslint.config.js`) across `**/*.{js,mjs,cjs,ts,jsx,tsx}`.
- `yarn typescript` — `tsc` type-check only (build emits nothing; `noEmit: true`).
- `yarn test` — Vitest watch mode (jsdom env, setup at `src/test/setup.ts`).
- `yarn test:unit` / `yarn test:integration` — scoped to `src/test/unit/` and `src/test/integration/`.
- Run a single test file: `yarn vitest run src/test/integration/ObjectsPage.spec.tsx` (add `-t "<name>"` to target a single `it/describe`).

Test spec glob in `vitest.config.ts` is `./src/test/**/*.spec.tsx` — colocated `*.test.ts` files outside `src/test/` are not picked up.

Required local configuration

Two files must exist before `yarn start` will work fully (both are gitignored):

1. `client/.env` — copy from `.env.template`. `REACT_APP_API_BASE_URL` and `REACT_APP_CMS_API_BASE_URL` (both with trailing slash) are the minimum needed for the app to consider local-env mode active (`config.hasLocalEnv`). Vite only exposes vars prefixed `REACT_` (see `envPrefix` in `vite.config.ts`).
2. `client/src/lib/pushClientEvent.ts` — copy from the committed `pushClientEvent.ts.template`. Without it, import resolution fails. The real file is intentionally ignored so analytics hooks stay site-specific.

When `.env` is missing or lacks those two URLs, the app calls `GET /env` on the hosting server instead. For local dev against the Vite server that call 404s, which is fine as long as `.env` is populated.

Architecture

Bootstrap flow

`src/index.tsx` → Redux `Provider` + `App`. `src/App.tsx` is a gate that blocks rendering until two RTK Query calls resolve:

1. `useGetEnvQuery` (skipped if `config.hasLocalEnv`) — hydrates the singleton `config.env` with `setServerConfig`.
2. `useGetAdvancedSearchConfigQuery` (skipped until env is loaded) — hydrates `config.advancedSearch`.

Only after both succeed (or `cacheViewerMode` lets AS config be skipped) does `App` mount `<AuthProvider>` (OIDC / AWS Cognito) → `<Router>` → `<LuxRoutes>`. In non-production envs, BugHerd is injected as a side-effect. **Do not read `config.env.*` from module top-level code** — it's a class instance that gets mutated during bootstrap.

Routing and entity dispatch

`src/features/common/LuxRoutes.tsx` defines the top-level routes:

- `/` and `/landing` → `<Landing>` (`features/landing/`)
- `/view/results/:tab/:subTab?` → `<ResultsPage>` (`features/results/`)
- `/view/*` (everything else) → `<RoutingComponent>` (`features/results/RoutingComponent.tsx`)
- `/content/:pageKey` → Drupal-backed CMS pages via `<CmsRoutingComponent>`
- `/view/advanced-search-config` → developer tool

`RoutingComponent` fetches the entity by stripping `/view/` from `pathname`, then dispatches to a page component based on `data.type`:

JSON-LD type	Page component
<code>HumanMadeObject</code> , <code>DigitalObject</code>	<code>features/objects/ObjectsPage</code>
<code>LinguisticObject</code> , <code>VisualItem</code>	<code>features/works/WorksPage</code>
<code>Set</code>	<code>features/set/SetsPage</code>
<code>Person</code> , <code>Group</code>	<code>features/personAndGroup/PersonAndGroupPage</code>
<code>Place</code>	<code>features/place/PlacePage</code>
anything in <code>searchTypes.concepts</code>	<code>features/concept/ConceptPage</code>
anything in <code>searchTypes.events</code>	<code>features/event/EventPage</code>

When adding a new entity type, wire it both in `RoutingComponent` and in the relevant arrays in `config/searchTypes.ts`.

State layer

`src/app/store.ts` composes three RTK Query APIs and seven slices:

- **APIs** (all in `src/redux/api/`): `mlApi` (data backend — entities, search, collections, related lists, timelines, estimates), `cmsApi` (Drupal JSON:API), `configApi` (the `/env` endpoint on the hosting server).
- **Slices** (`src/redux/slices/`): `advancedSearch`, `simpleSearch`, `currentSearch`, `facetSelection`, `helpTextKey`, `hierarchy`, `myCollections`.
- Shared axios-based `baseQuery` in `redux/api/baseQuery.ts` reads the current JWT from `config.currentAccessToken` and attaches it as a Bearer header. `getBaseUrl` is a function, not a string, because both base URLs get mutated by `setServerConfig` at runtime.

Use `useAppDispatch` / `useAppSelector` from `src/app/hooks.ts` (typed) instead of the bare `react-redux` hooks.

Layer conventions

- `src/features/<domain>/` — UI components for that domain. Mixed `styled-components` + `react-bootstrap` + plain CSS classes; `src/styles/` has cross-cutting theme/global styles (`global.ts`, `theme.ts`).
- `src/lib/parse/` — pure transforms from backend JSON-LD into view-model shapes. Grouped by `data/`, `search/`, `cms/`, `timeline/`. Reach for these before adding parsing logic inside a component.
- `src/lib/util/` — fetchers that are called from inside RTK Query endpoints (`fetchItems`, `fetchTimeline`, `fetchRelatedLists`, `fetchArchiveAncestors`, `fetchSearchEstimates`) plus URL/param helpers.
- `src/lib/hooks/` — shared React hooks (e.g. `useAuthentication` wraps `useAuth` from `react-oidc-context`).
- `src/types/` — shared TypeScript types. `data/` has JSON-LD entity shapes; `myCollections/` is feature-scoped. Keep the `I`-prefix convention (enforced by ESLint `naming-convention`).
- `src/config/` — static, build-time configuration and the `config` singleton. `config/advancedSearch/` and `config/myCollections/` hold larger sub-configs.

Auth

OIDC via `react-oidc-context` (AWS Cognito). `config.currentAccessToken` is the shared source of truth for the bearer token; `onSignIn` in `lib/auth/helper` populates it, and `baseQuery` reads from it. My-Collections features are gated on `REACT_APP_FEATURE_MY_COLLECTIONS` and protected with `PrivateRoute`.

Lint and style rules worth knowing

- ESLint is also run during `vite` builds via `@nabla/vite-plugin-eslint` — a lint error will fail the build, not just `yarn lint`.
- **Interface names must start with `I` + capital** (e.g. `IEntity`, `ISearchResults`). Non-conforming names will fail lint.
- `@typescript-eslint/explicit-function-return-type` is on (expressions are allowed to infer).
- `import/order` enforces grouped, blank-line-separated import groups: builtin → external → internal →

parent → sibling → index.

- `react/jsx-fragments: ['element']` — use `<React.Fragment>`, not `<>`.
- Unused-vars is explicitly **off** (both base and TS variants). Don't enable it without discussing — the repo currently relies on this.
- Prettier: no semicolons, single quotes, trailing commas, 80-col, 2-space. `yarn lint` runs Prettier as a warning.

Testing notes

- `src/test/setup.ts` registers `@testing-library/jest-dom` matchers and runs `miscMocks()` from `src/test/integration/utils/miscMocks` before each test — anything test-global belongs there.
- `nock` is used for HTTP mocking in integration tests; `redux-mock-store` for store mocks. Test data JSON lives in `src/test/data/`.
- Tests are `.spec.tsx` (not `.test.tsx`). The Vitest glob will not pick up other extensions.

Things that look wrong but are intentional

- `yarn.lock` appears in `git status` as both deleted and untracked because of case/line-ending churn on Windows — check the actual diff before committing.
- The `build` script does `mv .env _env || true` (see above) — this is why you may see a file called `_env` if a build was interrupted. Restore it to `.env`.
- `pushClientEvent.ts` is gitignored and recreated from a template per-environment. It is **not** a missing file.